ΤΕΧΝΙΚΗ ΑΝΑΦΟΡΑ ΕΡΓΑΣΙΑΣ

«Διαχείριση και σύνθεση ομάδων εργαστηρίων & Κατασκευή REST API»

Ονοματεπώνυμο: Δημήτρης Συμεωνίδης

Αριθμός Μητρώου: Π22165

Φεβρουάριος 2026

Τεχνικό Υπόβαθρο

Η υλοποίηση και των δύο θεμάτων έγινε σε ενιαίο codebase χρησιμοποιώντας το πλαίσιο Ruby on Rails, ακολουθώντας την αρχιτεκτονική MVC (Model-View-Controller).

- **Γλώσσα:** Ruby 3.2.10
- **Framework:** Rails 7.1.5
- **Βάση Δεδομένων:** SQLite3
- **Server:** Puma
- **Testing:** RSpec

Η εργασία έχει ανεβαστεί και στο github με οδηγίες εκτέλεσης (USER_MANUAL.md) και περαιτέρω πληροφορίες στο README.md

ΜΕΡΟΣ Α: ΘΕΜΑ 1 - Portal Εργαστηρίων

Στο πρώτο μέρος αναπτύχθηκε ένα αλληλεπιδραστικό Web Portal για τη διαχείριση και σύνθεση ομάδων. Η εφαρμογή εξυπηρετεί τους χρήστες μέσω Browser.

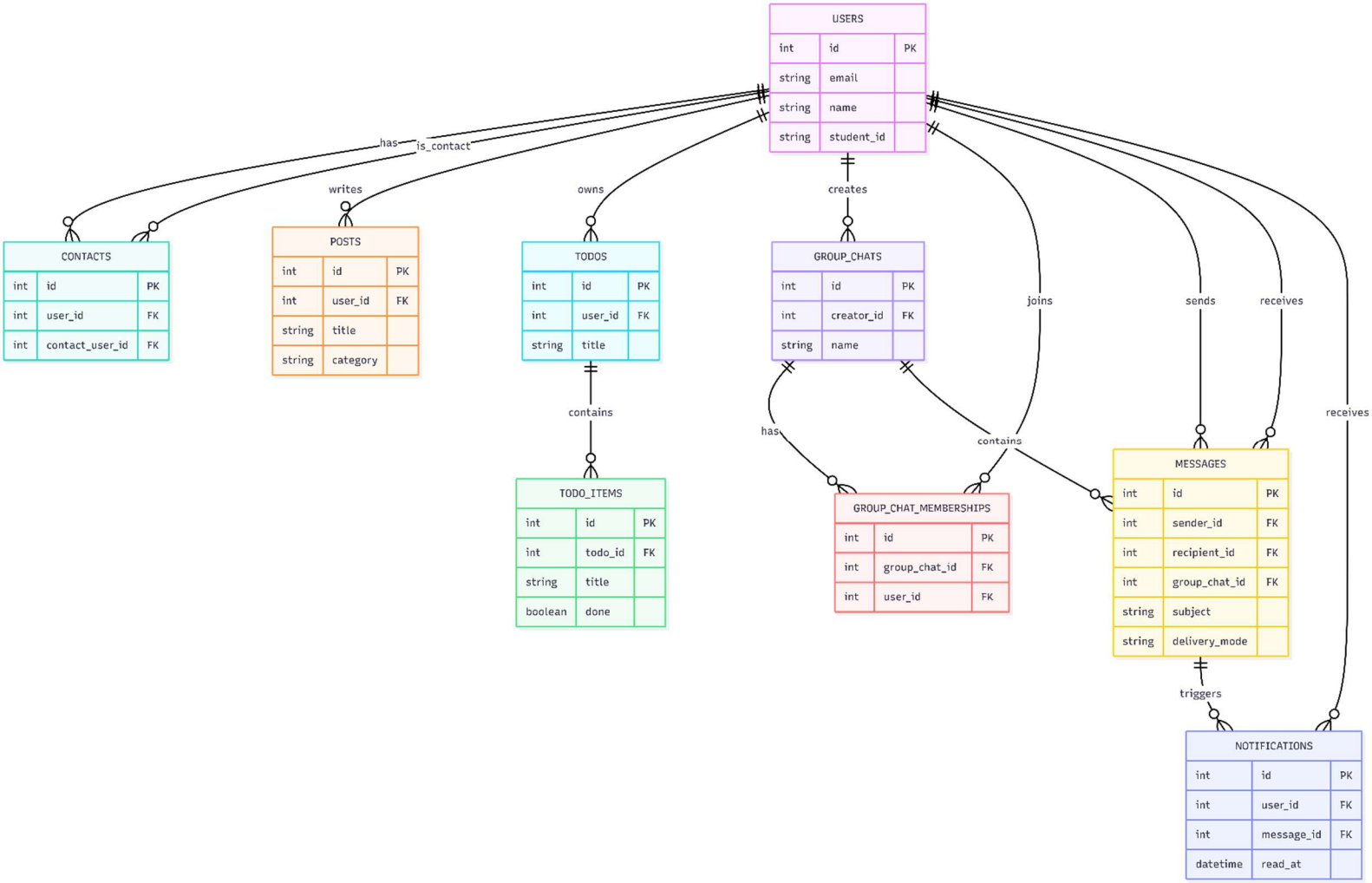
1. Αρχιτεκτονική & Βάση Δεδομένων

Η βάση δεδομένων σχεδιάστηκε για να υποστηρίζει τις κοινωνικές αλληλεπιδράσεις των φοιτητών.

Κυριότερες Οντότητες (Models):

- **User:** Ο κεντρικός πίνακας χρηστών. Συνδέεται με Posts, Messages, Contacts.
- **Post:** Οι αναρτήσεις των χρηστών.
- **Contact:** Πίνακας συσχέτισης (Self-Join) για τη δημιουργία λίστας επαφών μεταξύ χρηστών.
- **Message:** Υποστηρίζει τόσο προσωπικά μηνύματα (sender -> recipient) όσο και ομαδικά.
- **GroupChat & GroupChatMembership:** Μοντέλα για τη διαχείριση ομαδικών συνομιλιών.
- **Notification:** Πολυμορφικό μοντέλο για την ενημέρωση χρηστών.

ER Diagram:



2. Υλοποίηση Θέματος 1: Portal Εργαστηρίων

Στο πρώτο μέρος της εργασίας, αναπτύχθηκε το Portal κοινωνικής δικτύωσης. Ακολουθεί η αναλυτική παρουσίαση της υλοποίησης για κάθε ζητούμενο της εκφώνησης.

2.1 Λίστα Ελέγχου Υλοποίησης

✓ **Ζητούμενο 1: Δημοσιεύσεις & Αναζήτηση**

"Δυνατότητα να αποστέλλετε Posts, να αναζητείτε Posts και να τα κατηγοριοποιείτε."

Τεχνική Υλοποίηση: Για την κάλυψη αυτής της απαίτησης, δημιουργήθηκε το μοντέλο Post με πεδία title, body και category.

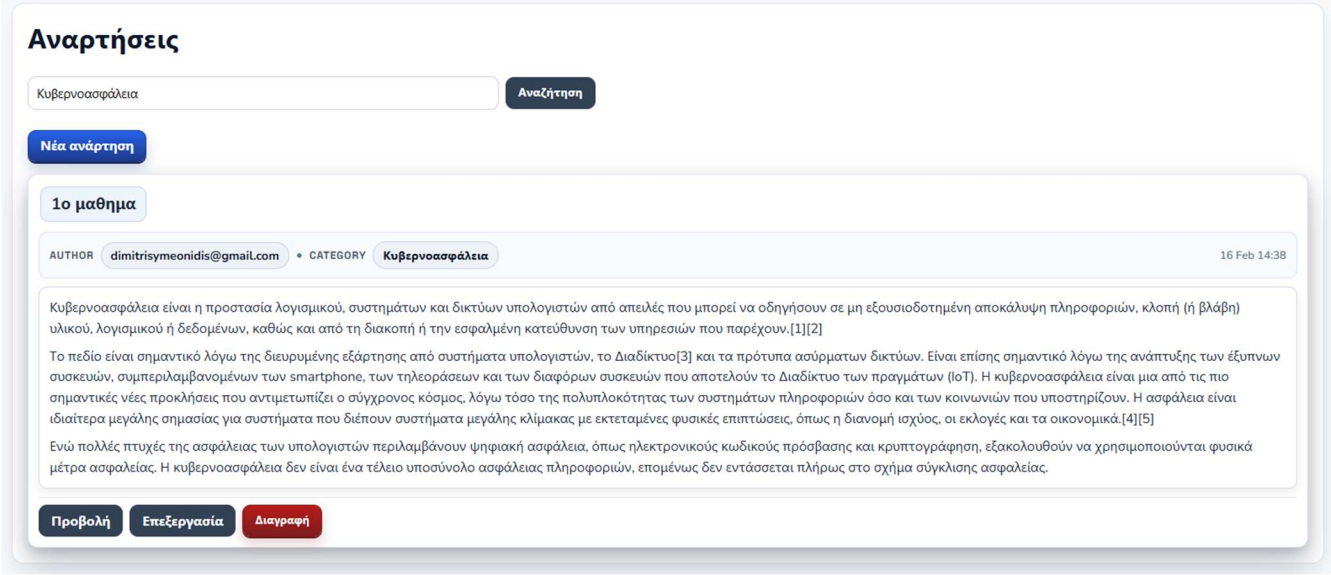
- **Δημιουργία:** Χρησιμοποιήθηκε ο PostsController ακολουθώντας τα RESTful πρότυπα του Rails (new, create).
- **Κατηγοριοποίηση & Αναζήτηση:** Αντί για απλή αναζήτηση, ο κώδικας εκτελεί SQL query που ελέγχει ταυτόχρονα τον τίτλο, το περιεχόμενο και την κατηγορία:

```
scope :search, ->(term) {
  where("title LIKE :q OR body LIKE :q OR category LIKE :q", q: "%#{term}%")
}
```

Με αυτόν τον τρόπο, ο χρήστης μπορεί να πληκτρολογήσει είτε θέμα είτε κατηγορία και να λάβει τα σχετικά αποτελέσματα.

Οδηγίες Χρήσης:

- Προβολή: Κάντε κλικ στο tab Αναρτήσεις στην κεντρική μπάρα πλοήγησης για να δείτε τη λίστα.
- Δημιουργία: Πατήστε το κουμπί Νέα Ανάρτηση, συμπληρώστε τη φόρμα και την κατηγορία και πατήστε Αποθήκευση
- Αναζήτηση: Στο πεδίο αναζήτησης που βρίσκεται πάνω από τη λίστα, πληκτρολογήστε μια λέξη κλειδί ή κατηγορία και πατήστε Αναζήτηση



✓ Ζητούμενο 2: Διαχείριση Επαφών

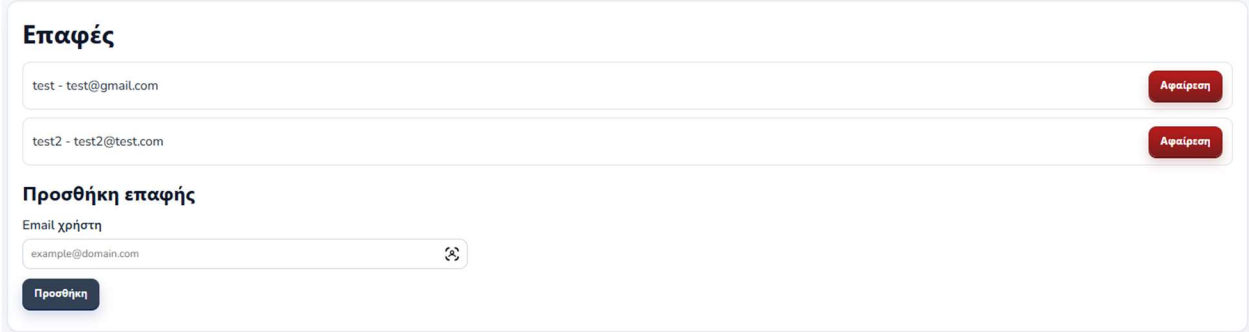
"Δυνατότητα να προστίθενται χρήστες ως προσωπικές επαφές ενός χρήστη."

Τεχνική Υλοποίηση: Η λειτουργία αυτή υλοποιήθηκε μέσω μιας αυτο-αναφορικής συσχέτισης.

- Δημιουργήθηκε το μοντέλο Contact που λειτουργεί ως ενδιάμεσος πίνακας (join table) συνδέοντας το User (χρήστης) με το ContactUser (η επαφή).
- Στον ContactsController, η ενέργεια create αποθηκεύει τη σχέση, επιτρέποντας στον χρήστη να διατηρεί λίστα με "φίλους" ή συνεργάτες για γρήγορη επικοινωνία.

Οδηγίες Χρήσης:

- **Λίστα Επαφών:** Επιλέξτε το tab Επαφές στο μενού για να δείτε τους φίλους σας.
- **Προσθήκη:** Υπάρχει πεδίο αναζήτησης χρηστών μέσω email. Συμπήρωνοντας email χρήστη πο υπάρχει στην βάση και πατώντας Προσθήκη, ο χρήστης προστίθεται στη λίστα σας.
- **Διαγραφή:** Πατήστε το κουμπί Διαγραφή δίπλα σε μια επαφή για να την αφαιρέσετε.



✓ **Ζητούμενο 3: Αυθεντικοποίηση**

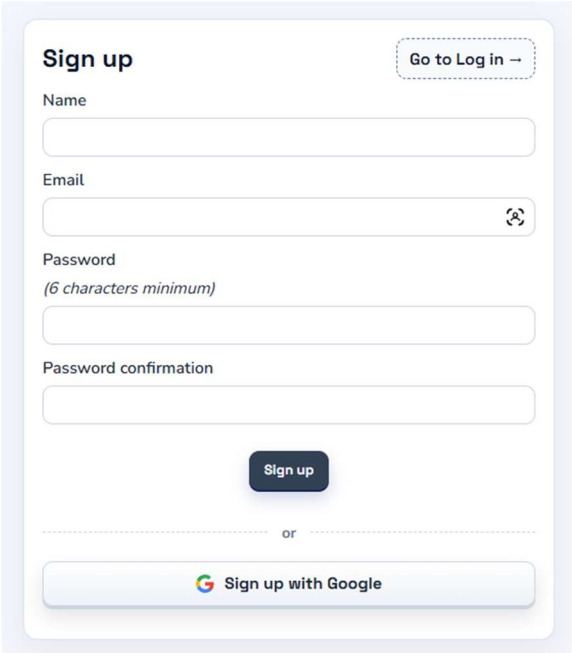
"Αυθεντικοποίηση με Facebook ή Google (3rd party) και Basic Authentication τοπικά."

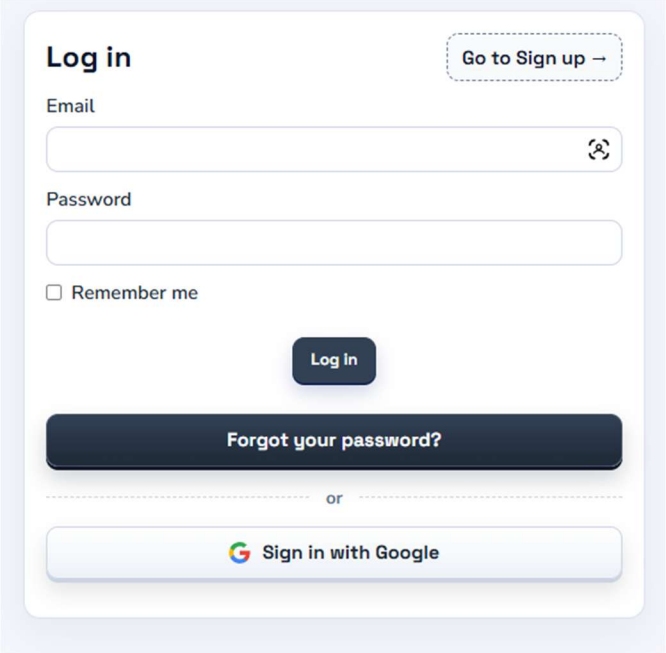
Τεχνική Υλοποίηση: Η ασφάλεια της εφαρμογής βασίστηκε στο gem Devise.

1. **Τοπική Είσοδος:** Χρησιμοποιήθηκαν τα modules :database_authenticatable και :registerable του Devise για εγγραφή με Email/Password.
2. **Google OAuth:** Ενσωματώθηκε το gem omniauth-google-oauth2. Δημιουργήθηκε ο Users::OmniauthCallbacksController, ο οποίος διαχειρίζεται την επιστροφή (callback) από την Google. Αν το email του χρήστη δεν υπάρχει στη βάση, ο λογαριασμός δημιουργείται αυτόματα εκείνη τη στιγμή.

Οδηγίες Χρήσης:

- **Είσοδος:** Στην αρχική οθόνη, πατήστε Log in. Μπορείτε να βάλετε Email/Password ή να πατήσετε το κουμπί Sign in with Google για άμεση σύνδεση.
- **Εγγραφή:** Αν δεν έχετε λογαριασμό, πατήστε Sign up και συμπληρώστε τα στοιχεία σας.
- **Έξοδος (Logout):** Για να αποσυνδεθείτε, πατήστε το κουμπί Logout που βρίσκεται αριστερά στην μπάρα πλοήγησης.





✓ **Ζητούμενο 4: Άμεσα Μηνύματα (Προσωπικά & Ομαδικά)**

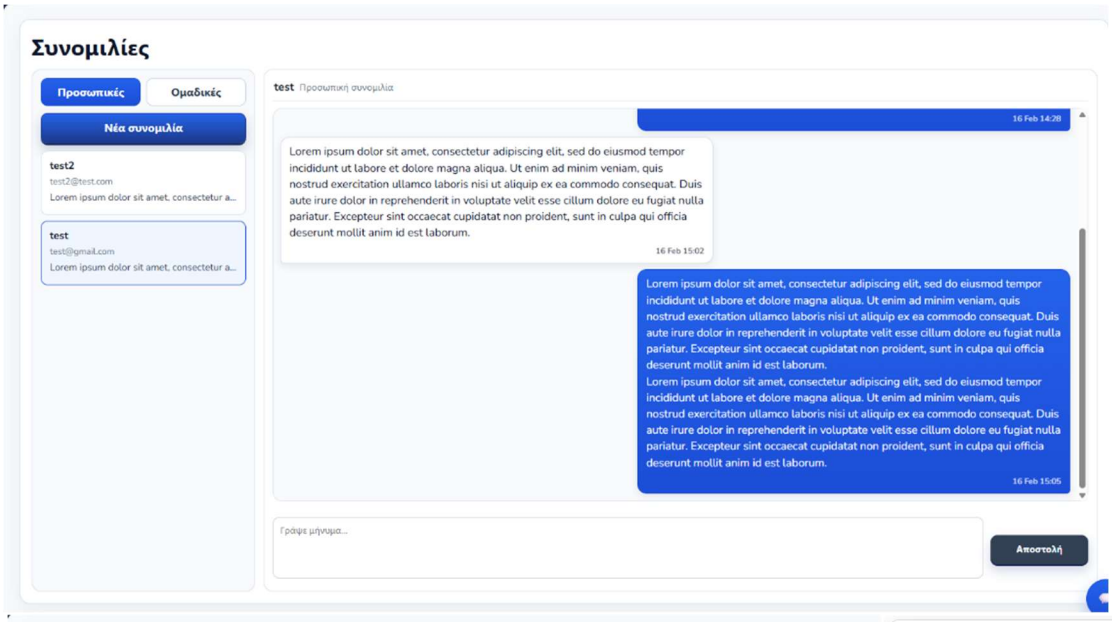
"Άμεσα μηνύματα (Popup παράθυρο), προσωπικά σε χρήστη ή προς ομάδα."

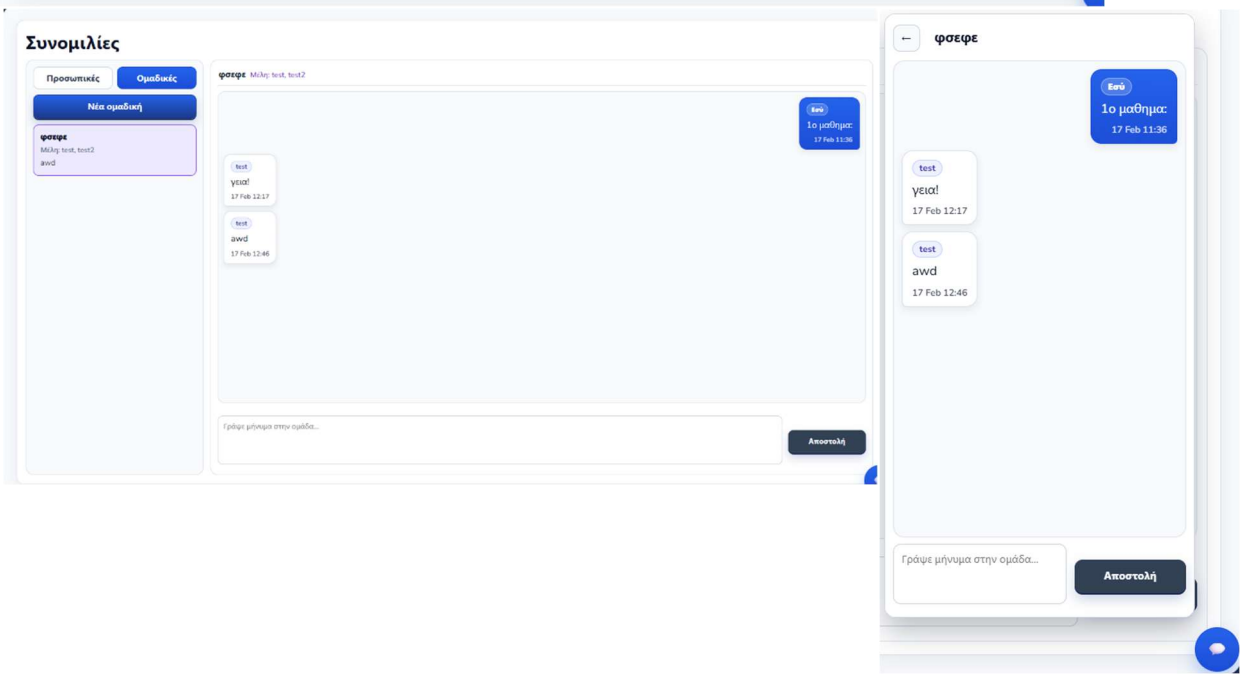
Τεχνική Υλοποίηση: Το σύστημα επικοινωνίας σχεδιάστηκε για να προσφέρει ευελιξία, υποστηρίζοντας δύο διαφορετικές μορφές διεπαφής (Views) που αντλούν δεδομένα από τον ίδιο MessagesController:

1. **Κεντρική Σελίδα Μηνυμάτων:** Μέσω της επιλογής Messages στο sidebar, ο χρήστης μεταφέρεται σε πλήρη οθόνη διαχείρισης (index.html.erb), όπου έχει πρόσβαση στο ιστορικό όλων των συνομιλιών του και μπορεί να διαχειριστεί τις ομαδικές συνομιλίες.
2. **Αναδυόμενο Παράθυρο (Floating Bubble / Popup)**: Για την άμεση επικοινωνία χωρίς διακοπή της πλοήγησης, υλοποιήθηκε λειτουργία Quick Chat. Πρόκειται για ένα αναδυόμενο παράθυρο (floating bubble) στο δεξιά μέρος της οθόνης, το οποίο επιτρέπει την γρήγορη αποστολή/λήψη μηνυμάτων ενώ ο χρήστης περιηγείται σε άλλες σελίδες (π.χ. ενώ βλέπει Posts).
3. **Νέα Συνομιλία:** Επιλέξτε παραλήπτη από τη λίστα επαφών ή πολλαπλούς χρήστες στο Ομαδική tab και ξεκινήστε την πληκτρολόγηση του μηνύματος σας.

Υποστηριζόμενοι τύποι συνομιλίας:

- Προσωπικά: Μοντέλο Message με sender_id και recipient_id.
- Ομαδικά: Μοντέλο GroupChat με GroupChatMemberships, όπου το μήνυμα συσχετίζεται με το ID της ομάδας.





- ✓ **Ζητούμενο 5: Ειδοποιήσεις**

"Άμεση ενημέρωση με Notification."

Τεχνική Υλοποίηση: Δημιουργήθηκε το μοντέλο Notification. Μέσω Callbacks στα μοντέλα (π.χ. after_create στο Message), δημιουργείται αυτόματα μια ειδοποίηση για τον παραλήπτη όταν λαμβάνει νέο μήνυμα ή αίτημα επαφής. Ο χρήστης βλέπει μόνιμα τον αριθμό των μη αναγνωσμένων ειδοποιήσεων στη μπάρα πλοήγησης.

ΜΕΡΟΣ Β: ΘΕΜΑ 2 - REST API (Todos)

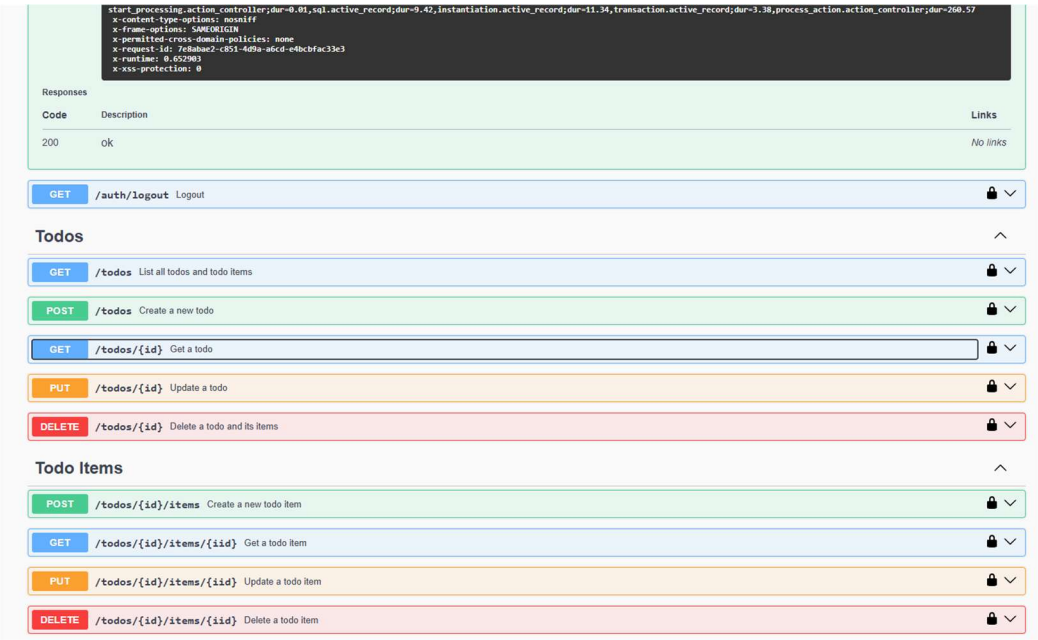
Στο δεύτερο μέρος της εργασίας, δημιουργήθηκε ένας μηχανισμός (API) που επιτρέπει σε άλλες εφαρμογές να διαχειρίζονται λίστες υποχρεώσεων (Todos). Ο σχεδιασμός έγινε έτσι ώστε το σύστημα να είναι αυτόνομο, ασφαλές και εύκολο στη χρήση.

3. Υλοποίηση REST API

Η ανάπτυξη έγινε με βάση τους σύγχρονους κανόνες του διαδικτύου OpenAPI 3.0, διασφαλίζοντας ότι τα δεδομένα μεταφέρονται σωστά και γρήγορα.

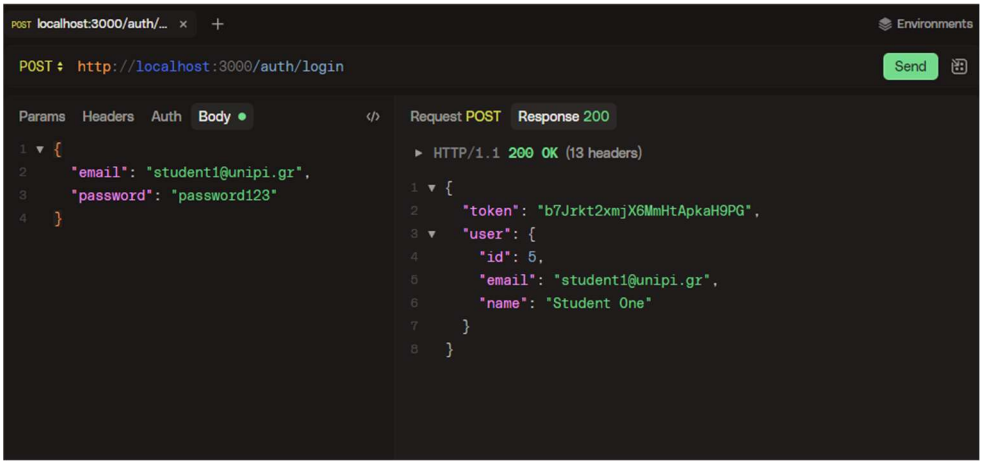
- ✓ **Οργάνωση Λιστών & Αντικειμένων**
  - **Πώς δουλεύει:** Το σύστημα οργανώθηκε σε δύο επίπεδα: τις Λίστες (Todos) και τα Περιεχόμενα (Items).
  - **Απλή Λογική:** Κάθε λίστα μπορεί να έχει πολλά αντικείμενα μέσα της. Ρυθμίστηκε το σύστημα έτσι ώστε αν κάποιος διαγράψει μια λίστα, να σβήνονται αυτόματα και όλα τα αντικείμενα που περιείχε, ώστε να μην μένουν «σκουπίδια» στη βάση δεδομένων.
- ✓ **Ασφάλεια με Ψηφιακό Κλειδί (Token)**
  - **Η Λειτουργία:** Για να προστατέψουμε τα δεδομένα των χρηστών, η πρόσβαση δεν είναι ελεύθερη.
  - **Κλειδί:** Κάθε χρήστης, αφού κάνει Login, λαμβάνει ένα μοναδικό Token (έναν κωδικό). Αυτό το κωδικό πρέπει να τον δίνει σε κάθε αίτημα που κάνει στο API. Αν το κλειδί δεν είναι σωστό, ο server απαγορεύει την πρόσβαση.
- ✓ **Τεκμηρίωση & Δοκιμές (Swagger)**
  - **Swagger UI:** Για να μπορεί οποιοσδήποτε να καταλάβει πώς δουλεύει το API, δημιουργήθηκε σελίδα οδηγιών Swagger. Η τεκμηρίωση και η δυνατότητα διαδραστικής δοκιμής όλων των endpoints είναι διαθέσιμη μέσω UI στη διεύθυνση: http://localhost:3000/api-docs

- **Έτοιμα Παραδείγματα:** Μέσα στη σελίδα αυτή, έχω βάλει έτοιμα δεδομένα (όπως email και κωδικούς). Έτσι, ο χρήστης δεν χρειάζεται να γράφει κώδικα με το χέρι. Μπορεί απλά να πατήσει ένα κουμπί και να δει το API να λειτουργεί ζωντανά.



✓ Αυτοματοποιημένοι Έλεγχοι (Tests)

Για βεβαιότητα ότι ο κώδικας δεν έχει λάθη, γράφτηκαν 34 αυτοματοποιημένα τεστ (RSpec). Αυτά τα τεστ ελέγχουν αυτόματα αν το σύστημα απαντάει σωστά όταν δίνουμε σωστά στοιχεία, αλλά και αν πετάει έξω κάποιον που προσπαθεί να μπει χωρίς το σωστό κλειδί. Επίσης, η συμβατότητα ελέγχθηκε και με το εργαλείο HTTPie για δοκιμές απευθείας από το τερματικό.



✓ **Πίνακας Endpoints**

Υλοποιήθηκαν όλα τα endpoints που ζητούνται στην εκφώνηση:

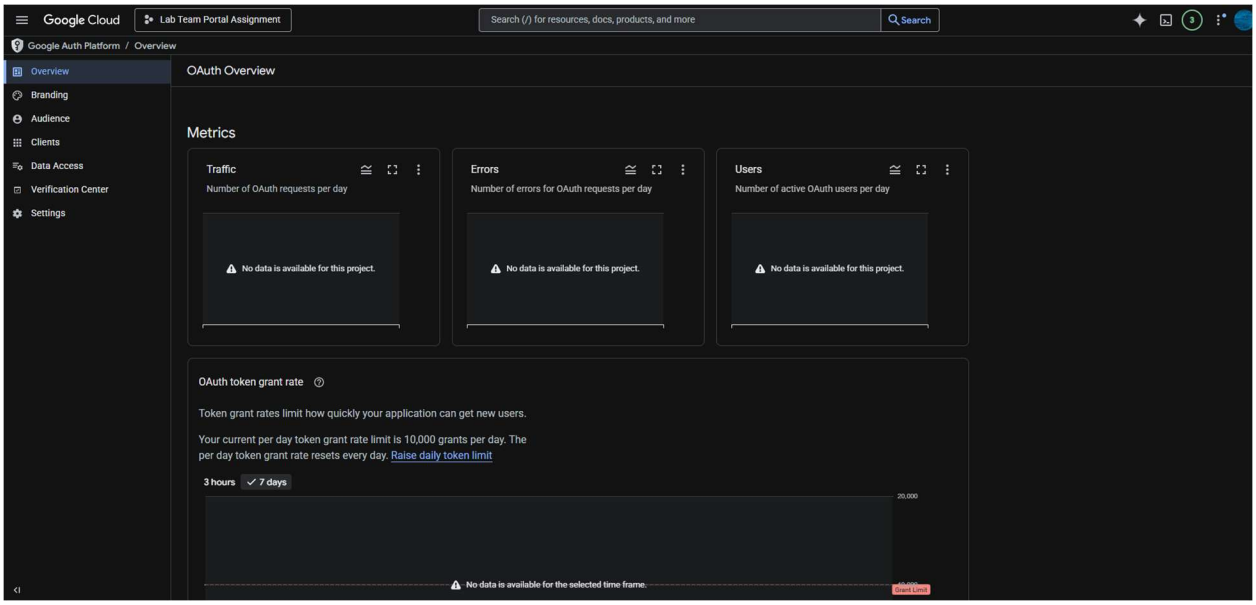
Endpoint	Functionality
POST /signup	Signup
POST /auth/login	Login
GET /auth/logout	Logout
GET /todos	List all todos and todo items
POST /todos	Create a new todo
GET /todos/:id	Get a todo
PUT /todos/:id	Update a todo
DELETE /todos/:id	Delete a todo and its items
GET /todos/:id/items/:iid	Get a todo item
POST /todos/:id/items	Create a new todo item
PUT /todos/:id/items/:iid	Update a todo item
DELETE /todos/:id/items/:iid	Delete a todo item

4. Ρυθμίσεις & Εγκατάσταση

Για να τρέξει η εφαρμογή σωστά, χρειάστηκαν μερικές βασικές ρυθμίσεις:

✓ 4.1 Σύνδεση με Google

Η εφαρμογή Ρυθμίστηκε στο Google Cloud ώστε να επιτρέπει στους φοιτητές να συνδέονται με το πανεπιστημιακό τους email. Δημιούργησα τα απαραίτητα ψηφιακά κλειδιά Client ID & Secret που επιτρέπουν στην εφαρμογή να επικοινωνεί με ασφάλεια με την Google.



✓ 4.2 Οδηγίες Εγκατάστασης & Εκτέλεσης

Βήμα 1 — Εγκατάσταση εξαρτήσεων:

```
bundle install
```

Βήμα 2 — Προετοιμασία βάσης δεδομένων:

```
ruby bin/rails db:setup
```

Η εντολή αυτή εκτελεί αυτόματα create, migrate και seed. Η βάση γεμίζει με δοκιμαστικά δεδομένα (χρήστες, posts, todos) για άμεση επίδειξη.

Βήμα 3 — Εκκίνηση server:

```
ruby bin/rails s
```

Η εφαρμογή ανοίγει στο: http://localhost:3000

Βήμα 4 — Εκτέλεση Tests (προαιρετικό):

```
bundle exec rspec spec/requests/api
```

Εκτελεί τα 34 αυτοματοποιημένα tests του API.

✓ **4.3 Demo Λογαριασμοί**

Για τη δοκιμή της εφαρμογής, δημιουργούνται αυτόματα (μέσω seeds) οι παρακάτω λογαριασμοί:

Email	Password	Σημείωση
student1@unipi.gr	password123	Έχει ήδη posts, todos, επαφές
student2@unipi.gr	password123	Για δοκιμή chat & επαφών

Τα ίδια credentials χρησιμοποιούνται τόσο για το Web Portal (login μέσω browser) όσο και για το REST API (endpoint POST /auth/login).

---

5. Εμφάνιση & Χρήση (UI)

Η εφαρμογή σχεδιάστηκε για να είναι απλή:

- **Προσαρμόσιμη:** Δουλεύει σωστά τόσο σε υπολογιστή όσο και σε κινητό.
- **Ενημερώσεις:** Βγάζει άμεσα μηνύματα επιτυχίας για να ξέρει ο χρήστης ότι όλα πάνε καλά.